

PROJECT REPORT

Insurance Policy Management System (IPMS)

ASP.NET Core Web API using 3-Tier Architecture

US Insurance Market Domain

1. Introduction

In today's insurance industry, especially in the United States market, insurance companies handle a very large volume of customers, policies, payments, and claims daily. Most traditional insurance systems are tightly coupled, difficult to maintain, and not scalable enough to support modern business requirements such as API integrations, mobile apps, and partner systems.

This project, Insurance Policy Management System (IPMS), is designed to simulate a real-time enterprise insurance application developed in a CMMI Level 3 compliant IT organization. The main objective of this project is to train junior developers on end-to-end application development using ASP.NET Core Web API, 3-Tier Architecture, and SQL Server with Stored Procedures.

The project covers the complete software development lifecycle starting from requirement understanding, database design, API development, business logic implementation, security, logging, and final documentation.

2. Purpose of the Project

The purpose of this project is to:

- Provide hands-on experience on real-world enterprise application development
- Train juniors on layered architecture and clean coding practices
- Understand the US insurance business domain
- Learn database design with normalization and stored procedures
- Implement secure REST APIs using ASP.NET Core
- Follow structured development similar to MNC insurance projects

This project is not a simple CRUD application, but a business-driven system with real workflows and validations.

3. Business Domain Overview – US Insurance Market

Insurance companies in the US mainly deal with the following types of insurance:

- **Health Insurance**
- **Life Insurance**
- **Term Insurance**
- **Group Insurance**

Key stakeholders involved in insurance operations:

- **Customer – Buys insurance policy and raises claims**
- **Insurance Agent – Helps customer in policy selection**
- **Underwriter – Verifies and approves policies and claims**
- **Admin – Manages master data and system configuration**

Typical insurance lifecycle:

1. **Customer registration**
2. **Policy quotation**
3. **Policy issuance**
4. **Premium payments**
5. **Claim initiation**
6. **Claim approval or rejection**
7. **Policy closure or renewal**

This project is designed to cover all the above stages.

4. Project Scope

4.1 In Scope

- **ASP.NET Core Web API backend**
- **3-Tier Architecture implementation**
- **SQL Server database design**
- **Stored procedures for all database operations**
- **JWT-based authentication**
- **Role-based authorization**
- **Logging and audit trail**
- **End-to-end insurance workflow**

4.2 Out of Scope

- **UI / Frontend development**
- **Real payment gateway integration**
- **Third-party insurance services**

5. Technology Stack

Component	Technology
Backend Framework	ASP.NET Core Web API
Language	C#
Architecture	3-Tier Architecture
Database	SQL Server
Data Access	ADO.NET / Dapper
Authentication	JWT
API Documentation	Swagger
Logging	Serilog
Version Control	Git

6. System Architecture

The application follows 3-Tier Architecture, which is mandatory in enterprise projects.

6.1 Presentation Layer (API Layer)

- Exposes REST APIs
- Accepts HTTP requests
- Validates input data
- Sends proper HTTP responses
- Does NOT contain business logic

6.2 Business Logic Layer

- Contains all insurance rules
- Handles policy eligibility checks
- Calculates premium and penalties
- Controls claim approval workflow
- Acts as middle layer between API and database

6.3 Data Access Layer

- Responsible for database communication
- Calls stored procedures only
- Maps database results to models
- Handles SQL exceptions

7. Functional Modules

7.1 User and Role Management Module

This module handles authentication and authorization of users.

User Roles:

- Admin

- **Insurance Agent**
- **Underwriter**
- **Customer**

Features:

- **User registration**
- **Login with JWT token**
- **Role-based API access**
- **Activate / deactivate users**

7.2 Customer Management Module

This module manages customer details.

Features:

- **Create customer profile**
- **Update customer information**
- **Maintain customer address**
- **View customer policies**
- **Deactivate customer**

Each customer is uniquely identified using SSN, as per US insurance standards.

7.3 Insurance Product Management Module

This module manages insurance products offered by the company.

Features:

- **Create insurance products**
- **Define coverage amount**
- **Set base premium**
- **Activate / deactivate products**

- **Assign policy type**

Examples:

- **Health Insurance – Gold Plan**
- **Life Insurance – Term Plan**

7.4 Policy Management Module

This is the core module of the application.

Features:

- **Policy quotation**
- **Policy issuance**
- **Policy renewal**
- **Policy cancellation**
- **Policy status tracking**

Policy Status:

- **Quoted**
- **Active**
- **Expired**
- **Cancelled**

Business rules are strictly implemented in the Business Layer.

7.5 Premium Payment Module

This module manages premium payments.

Features:

- **Generate premium schedule**
- **Record payments**
- **Handle late payments**

- View payment history
- Apply penalty rules

Payment gateway is mocked for training purpose.

7.6 Claims Management Module

This module handles insurance claims.

Features:

- Raise insurance claim
- Upload claim documents (metadata only)
- Assign underwriter
- Approve or reject claim
- Track settlement status

Claim approval is done only by Underwriter role.

8. Database Design

The database is designed using normalized relational structure.

8.1 Major Tables

Security Tables

- Users
- Roles
- UserRoles

Customer Tables

- Customers
- CustomerAddress

Insurance Tables

- InsuranceProducts
- Policies

Payment Tables

- PremiumPayments

Claim Tables

- Claims
- ClaimDocuments
- ClaimStatus

Audit Tables

- AuditLogs
- ErrorLogs

8.2 Database Design Principles

- Third Normal Form (3NF)
- No duplicate data
- Proper primary and foreign keys
- Audit fields in all tables
- Soft delete using IsActive flag

9. Stored Procedures Strategy

In this project:

 Direct table access is not allowed

 Only stored procedures must be used

Stored Procedure Categories:

- CRUD procedures
- Business logic procedures

- **Reporting procedures**

Examples:

- **sp_AddCustomer**
- **sp_CreatePolicy**
- **sp_ProcessPremiumPayment**
- **sp_RaiseClaim**
- **sp_ApproveClaim**

Stored procedures must:

- **Use transactions**
- **Handle errors**
- **Return status codes**

10. API Design Guidelines

- **RESTful design**
- **Meaningful URLs**
- **Proper HTTP status codes**
- **Input validation**
- **Secure endpoints**

Sample APIs:

Customer APIs

POST /api/customers

GET /api/customers/{id}

PUT /api/customers/{id}

Policy APIs

POST /api/policies/quote

POST /api/policies/issue

Claims APIs

POST /api/claims

PUT /api/claims/{id}/approve

11. Security Implementation

- **JWT token authentication**
- **Role-based authorization**
- **Secure password hashing**
- **Token expiration handling**

12. Logging and Auditing

Logging:

- **All exceptions logged**
- **API errors captured**
- **Database errors recorded**

Auditing:

- **Policy creation**
- **Claim approval**
- **Payment transactions**

13. Development Plan (Weekly Milestones)

This project is planned for 8–10 weeks with structured milestones:

- **Week 1: Requirement analysis**
- **Week 2: Database design**
- **Week 3: SQL implementation**
- **Week 4: Stored procedures**

- **Week 5: Project setup**
- **Week 6: Core modules**
- **Week 7: Policy module**
- **Week 8: Claims module**
- **Week 9: Payments & logs**
- **Week 10: Testing & documentation**

14. Testing Strategy

- **Unit testing (basic)**
- **API testing using Postman**
- **Negative test cases**
- **Role-based access testing**

15. Deliverables

- **Complete source code**
- **SQL scripts**
- **ER diagram**
- **API documentation**
- **Final project report**

16. Evaluation Criteria

Area	Weight
Architecture	25%
Database Design	25%
Business Logic	20%

Area	Weight
Code Quality	15%
Documentation	15%

17. Conclusion

The Insurance Policy Management System project provides trainees with real industry exposure to enterprise application development. By completing this project, trainees will gain confidence in:

- Working on real business domains**
- Designing scalable applications**
- Writing maintainable code**
- Understanding corporate development standards**

This project closely reflects the expectations of US insurance clients and Indian IT service companies, making trainees job-ready.